Lesson 1 - Bare Metal

1.1 The Linker Script

- Compilers turn C code into raw machine instructions called object files (.o). The compiler does not know where these instructions will live in physical RAM.
- A **Linker** stiches together these object files into a single binary. Normal programs rely on the host OS to load them into memory dynamically. We must provide a blueprint--a **Linker Script**--to tell the linker exactly which physical memory addresses our kernel will occupy.

The Blueprint

- The compiler manufactures the bricks (code and data). The linker script is the blueprint that dictates exactly which coordinates on the property (RAM) the rooms must be built.
- **Our kernel must start** precisely at the 1 Megabyte mark in physical RAM. This is because the addresses below 1MB are a historical minefield containing the BIOS, VGA text buffers, and memory-mapped hardware. If we load our kernel there, we will overwrite hardware state and instantly crash.

Writing the Script

- We will create a file named `linker.ld`. It uses a specific declarative syntax with the following rules:
  1. **The Entry Point:** You must tell the linker the name of the very first function that will execute. Use the command `ENTRY(symbol_name)`. We will name our entry function `_start` (to be written in Assembly later).
  2. **The Sections Block:** Open a block called `SECTIONS {...}`. Everything else goes inside this block.
  3. **The Location Counter:** Inside the block, the period character `.` represents the current memory address. Set this to 1MB. (Use the hexadecimal representation of 1,048,576 bytes.)
  4. **The Text Section (Code):** Define a section named `.text`.
     - **Rule:** Every section must be aligned to 4 Kilobytes to match the page size of modern CPU architectures. The syntax for a section is: `.section_name ALIGN(4K) : {...}`
     - **Contents:** Inside `.text`, you must tell the linker what to put there. We use the `*` wildcard to mean "from all files." You must place `*(.multiboot)` first, followed by `*(.text)`.
  5. **The Read-Only Data Section:** Define a `.rodata` section. Align it to 4K. Inside, place `*(.rodata)`. This holds string literals like "Hello World".
  6. **The Data Section:** Define a `.data` section. Align to 4K. Inside, Inside, place `*(.data)`. This holds initialized global variables.
  7. **The BSS Section**: Define a .bss section. Align it to 4K. Inside, place `*(COMMON)` and then `*(.bss)`. This holds uninitialized variables and, crucially, the memory we will use for our kernel's Stack.

Pitfalls:

- **Misplacing the Multiboot Header:** The bootloader (GRUB or QEMU) will only scan the first 8 kilobytes of your binary looking for a "Magic Number" to confirm it is an OS. If you do not put `*(.multiboot)` at the absolute top of the .text section, the bootloader will not find it, and the system will refuse to boot.
- **Missing Semicolons:** Setting the location counter requires a semicolon (e.g. `VAR = VALUE`)'

1.2 The Multiboot Header & Assembly Boot Stub

- You cannot boot to a C program directly. C relies on a stack to handle function calls and local variables. When the CPU first powers on and the bootloader hands over control, **there is no stack.** If your first instruction is a C function, the CPU will try to push a return address to a non-existent stack and instantly trigger a **triple fault** (rebooting the machine).
- Therefore, the very first piece of code must be written in Assembly. Its only jobs are:
  1. Provide the "VIP Pass" (Multiboot Header) so the bootloader recognizes the OS.
  2. Set up the Stack Pointer.
  3. Call the C kernel.
  4. Catch the CPU if the C kernel ever crashes or returns.

The bouncer and the Scaffolding

- Think of the bootloader (GRUB) as a bouncer. It scans the first 8 Kilobytes of your binary. If it does not find the exact sequence of "Magic Numbers" defined by the Multiboot Specification, it rejects your OS. Once you pass the bouncer, you must build the scaffolding (the stack) before you can pour the concrete (executing the C code).

Writing `boot.s`

You will write a GNU Assembly file targeting the 32-bit x86 architecture (i686). We use x86 because the standard VGA text buffer is an x86 IBM PC standard.
Create a file named `boot.s`. Follow these specifications:

1. Define the Multiboot Constants:
   - Declare a constant `MBALIGN` set to `1 << 0` (this tells the bootloader to align modules on page boundaries).
   - Declare a constant `MEMINFO` set to `1 << 1` (this asks the bootloader to provide a memory map).
   - Declare a constant `FLAGS` set to `MBALIGN | MEMINFO`.
   - Declare the `MAGIC` constant. The Multiboot1 specification requires this to be exactly `0x1BADB002`.
   - Declare the `CHECKSUM`. The specification requires that `MAGIC + FLAGS + CHECKSUM = 0`. Therefore, `CHECKSUM` must equal `-(MAGIC + FLAGS)`.
2. The Multiboot Section:
   - Open `.section .multiboot`
   - Use `.align 4` to ensure it is 32-bit aligned.
   - Output the three constants sequentially using `.long MAGIC`, `.long FLAGS`, and `.long CHECKSUM`.
3. The Stack Section:
   - Open `.section .bss`
   - Use `.align 16` (the x86 system V ABI requires a 16-byte aligned stack).
   - Create a label `stack_bottom:`.
   - Allocate 16 Kilobytes of empty space using `.skip 16384`
   - Create a label `stack_top:`. The stack grows *downwards* on x86, so `stack_top` is the address we will give to the CPU.
4. The Entry Point (The Code):
   - Open `.section .text`

- Declare the entry point global: `.global _start`
- Create the label `_start:`
- Move the address of `stack_top` into the CPU's stack pointer register (esp). in GNU assembly, the syntax is `mov $stack_top, %esp`
- Call your C function using `call kernel_main`

5. The Safety Net:
   - If `kernel_main` ever returns, the OS has ended. We must safely halt the CPU.
   - Disable interrupts: `cli` (Clear interrupts)
   - Halt the CPU: hlt (Halt).
   - Create a local label (e.g. 1:) and unconditionally jump back to it (jmp 1b) to create an infinite loop. This prevents the CPU from executing random memory if a Non-maskable interrupt (NMI) wakes it from the halt state.

Pitfalls:

- **Wrong Magic Number:** If you mistype the value for `MAGIC`, the emulator will say "No bootable device found."
- **Stack Direction:** x86 stacks grow downwards (from higher memory addresses to lower). If you set `%esp` to `stack_bottom`, the very first push will write into unallocated memory, destroying your data and crashing the kernel.
- **Missing Checksum Logic:** The bootloader mathematically verifies the checksum. If your formula is slightly off, the bootloader will reject the kernel.
  Extra Lesson 1.1 - The 'Awakening' Sequence

1.3 Writing the C Kernel (`kernel.c`)

You are entering a "freestanding" **C environment.** This means there is no underlying operating system to provide you with a Standard Library. Functions like `printf`, `malloc`, and `exit` do not exist. To output text, we must bypass software entirely and write directly to a hardware-mapped memory address known as the **VGA Text Buffer**.

The Hardware Lite-Brite

Think of your monitor in textmode as a grid of 80 columns and 25 rows (2,000 slots total). This grid is physically wired to physical RAM starting at address `0xB8000`. If you write data into slot 0, it instantly appears in the top-left corner of your monitor. If you write data into slot 1, it appears in the second column, and so on.

The Rules of VGA Memory:

Each character on the screen takes up exactly 2 bytes (16bits) in RAM:

- **Byte 1 (Lower 8 bits)**: The ASCII character (e.g., `'C'`).
- **Byte 2 (Upper 8 bits)**: The color attribute. (e.g., Light green text on a black background is represented by the hexadecimal value 0x0A).
  Therefore, to print a green 'C', we must write the 16-bit value `0x0A43` directly to address `0xB8000`.
  Extra Lesson 1.3 - Memory-Mapped IO (VGA Buffer + More)

Writing `kernel.c`

Create a file named `kernel.c`. You will write the C code using the following rules:

1. **The Entry Function:** Define a function named `void kernel_main(void)`. This exact name is what was used in `boot.s` that explicitly calls it.
2. **The Screen Pointer:** Inside the function, declare a pointer that points to the start of the VGA buffer.
   - *Rule:* Each slot on the screen requires exactly 16 bits. Therefore, your pointer must be an `unsigned short*`, which is 16 bits in standard C.
   - *Syntax*: Cast the hexadecimal address to a pointer: `unsigned short* terminal_buffer = (unsigned short*)0xB8000;`
3. **The Bitwise Math (Constructing the Character)**: You must construct a 16-bit value to put into the buffer.
   - To combine the lower and upper bits, use bitwise left-shift (`<<`) on the color to push it to the upper 8 bits, then bitwise OR (`|`) it with the character: (`0x0A << 8) | 'X'` .
4. **Writing to Memory:** Assign that constructed 16-bit value to the first index of your pointer (`terminal_buffer[0]`).
5. **The Infinite Loop:** Add a `while(1)` loop at the very end of the function. Even though we have a safety net in assembly, a well-designed kernel should never attempt to return.

Pitfalls

- Including <stdio.h>: Your muscle memory will tempt you to include standard headers. If you do, the compiler will look for Linux-specific files and compilation will fatally fail.
- **Using a `char*` Pointer:** A `char` is only 8 bits. If you point a `char*` at the VGA buffer and try to write to it, you will overwrite the ASCII character but leave the color byte untouched, resulting in misaligned memory and a garbled screen. You must use `unsigned short*`.

1.4 The Build System (`Makefile`)

Extra Lesson 1.2 - How Make Works
We now have the three raw materials we need: the memory blueprint (`linker.ld`), the CPU scaffolding (`boot.s`), and the OS logic (`kernel.c`). It is time to assemble them.

- Compiling a bare-metal kernel is fundamentally different from compiling a standard C program. By default, the `gcc` compiler assumes you are building a program for the host OS (Linux). It will invisibly inject code to set up exception handling, link the standard c library (`libc`), and compile it for a 64-bit architecture.
- If those invisible additions make it into our kernel, the CPU will immediately crash upon booting because it will look for a Linux environment that does not exist. We must use specific **compiler flags** to strip away all host OS assumptions.

The Assembly Line

Think of `make` as a factory manager. The `Makefile` is the assembly line manual. You define the final product (`casino.bin`), the parts required to build it (`boot.o` , `kernel.o`), and the exact machinery settings (compiler flags) needed to forge those parts.

Writing the `Makefile`

Because the VGA text buffer is an x86 standard, we are building a 32-bit kernel. We will force your 64-bit Linux compiler to cross-compile to 32-bit x86.
Create a file named `Makefile`. Follow these rules to construct it:

1. **Define the Tools & Flags:** At the top of the file, define these variables.
   - `CC = gcc`
   - `AS = as`
   - `CFLAGS = -m32 -ffreestanding -O2 -Wall -Wextra -fno-pie -fno-stack-protector`
     - *Why?* `-ffreestanding` tells GCC we have no standard library. `-fno-pie` and `-fno-stack-protector` stop GCC from injecting security features that require a host OS.
   - `ASFLAGS = --32`
   - `LDFLAGS = -m elf_i386 -T linker.ld -nostdlib`

- **Define the Final Product:** * Target: `casino.bin`
  - Dependencies: `boot.o kernel.o`
  - Command: `ld $(LDFLAGS) -o casino.bin boot.o kernel.o`
- **Define the Object Rules:**
  - Target: `boot.o`, Dependency: `boot.s`
    - Command: `$(AS) $(ASFLAGS) boot.s -o boot.o`
  - Target: `kernel.o`, Dependency: `kernel.c`
    - Command: `$(CC) $(CFLAGS) -c kernel.c -o kernel.o`
- **Define the Emulator Run Target:**
  - Target: `run`, Dependency: `casino.bin`
  - Command: `qemu-system-i386 -kernel casino.bin`
- **Define a Clean Target:**
  - Target: `clean`
  - Command: `rm -f *.o *.bin`

Pitfalls

- **The TAB rule:** In a `Makefile`, the commands underneath a target **must** be indented with a single tab character, not spaces. If you use spaces, `make` will immediately throw a `missing separator` error and halt.
- **Missing Multilib:** Because our host is 64-bit Debian, and we are compiling 32-bit x86 code, standard `gcc` will fail unless the 32-bit libraries are installed.